readme

Page

web-doscom Backend

Backend API for Dinus Open Source Community (DOSCOM)

PLEASE REPORT ANY BUGS IMMEDIATELY

Requirements

- Go 1.18+
- PostgreSQL
- air (for live reload, optional)
- swag (for Swagger docs)

Setup

1. **Clone the repo**

```
    git clone https://github.com/Dinus-Open-Source-Community/web-doscom.git
    cd web-doscom/BackEnd
```

2. **Configure environment**

- Copy `.env.example` to `.env` and edit DB credentials as needed.

3. **Run database migrations**
   - Use the SQL files in `migrations/` to set up your database tables.
     - `go run ./cmd/migrate/main.go up`
4. **Install dependencies and tools**

```
    go mod tidy
```

5. **Generate Swagger docs**

```
    swag init -g cmd/api/main.go -o docs
```

6. **Run the server (with live reload)**
   - with air -> hot live reload (menyusul)
   - manual

    - run command `go run cmd/api/main.go`
7. akses swagger
    - akses `http://localhost:3000/swagger/index.html#/`

How to run BackEnd

- buka terminal
- masuk ke dir -> backend `web_doscom/BackEnd`
- run command `go run cmd/api/main.go`
- akses swagger di localhost `http://localhost:3000/swagger/index.html#/`
- selamat testing

USER

1. super admin -> create akun mandiri -> create another superAdmin
2. super admin
    1. bikin akun para koor dan bph -> dengan username & password template
    2. semua crud -> aku developper sak gelem ku ah
    3. ubah role users
3. para koor dan bph
    1. bikin akun anggota nya masing masing -> dengan username & password template
    2. bisa hapus akun para anggota nya
4. pengurus
    1. update akun -> password email dll
    2. bisa update data diri si bagian pengurus

EndPoint localhost:3000/

- Auth
    - POST /api/v1/login
        - LoginRequest
            - email -> required
            - password -> required
        - response
            - development -> pakai ini sekarang

```
    "message":       "Login success, nasi padangnya sebungkus bolo",
    "token":         accesToken,
    "refresh_token": tokenHash,
```

            - production

```
    "message:": "login success bolo, nasi padang satu bungkus",
```

    - POST /api/v1/auth/logout
        - need auth/login first

- POST /api/v1/auth/refresh
  - cookie
    - "RefreshToken"
  - response
    - error

```
"error":   err.Error(),
"message": "refresh token invalid or expired",
```

    - succes
      - production

```
"message": "refresh token success created",
```

      - development

```
"message": "refresh token success",
"token":   newAccessToken,
```

- POST /api/v1/register
  - endpoint for create user -> still dont know for what -> **TIDAK DI PAKAI**

- User
  user management
  - POST /api/v1/user
    - create user -> role based -> jadi yang bisa akses hanya role
      - SuperAdmin
      - Koor
      - BPH
    - register request
      - username
      - email -> required
      - password
      - role -> only for role SuperAdmin -> required
      - fullname -> required
    - response
      -
  - POST /api/v1/user/admin
    - create another super admin
    - CAN ONLY ACCES BY SUPERADMIN
    - register request
      - username
      - email -> required
      - password
      - role
      - fullname -> required
  - GET /api/v1/user/:id
    - get user by id user
    - Roles that can access this endpoint
      - SuperAdmin
      - Koor
      - BPH
    - param -> id user

- GET /api/v1/user
  - get all user
  - roles that can acces these resource
    - SuperAdmin
    - Koor
    - BPH
- PUT /api/v1/user/:id
  - update user by id
  - roles that can acces these resource
    - SuperAdmin
    - Koor
- DELETE /api/v1/user/:id
  - delete user by id and base on role
  - roles that can acces these resource
    - SuperAdmin -> can delete all user (i have the powerrr)
    - Koor -> koor can only delete user from his division

- Pengurus
  data diri pengurus
  - POST /api/v1/pengurus/division/:division
    - get all pengurus data diri
    - public route
  - GET /api/v1/admin/pengurus/:id
    - get pengurus by id
    - role that can access these endpoint
      - all roles
  - POST /api/v1/admin/pengurus
    - create data pengurus diri pengurus
    - all roles can access
    - create request
      - id_user
      - photo_url
      - email
      - divisi -> required
      - name -> required -> min=2 max=150
      - position -> required
      - sosmed
      - period -> required -> max=50
  - GET /api/v1/admin/pengurus/:id
    - get data pengurus by id
    - all roles can access
  - PUT /api/v1/admin/pengurus/:id
    - update data pengurus by id
    - all roles can acces
    - update request
      - email
      - divisi
      - name
      - sosmed

        - period -> string -> e.g: 2024
        - position
        - url_asset
    - valid data

```
// daya yang valid utnuk inputan
     var ValidDivisi = map[string]bool{
          "bph":      true,
          "pemro":    true,
          "jaringan": true,
          "medcrev":  true,
          "data":     true,
     }

     var ValidPosition = map[string]bool{
          "ketum":        true,
          "sdm":          true,
          "pr":           true,
          "pm":           true,
          "pmAng":        true,
          "KoorPemro":    true,
          "KoorJaringan": true,
          "KoorMedcrev":  true,
          "KoorData":     true,
          "sekum":        true,
          "bendum":       true,
          "sekAng":       true,
          "bendAng":      true,
          "PemroAng":     true,
          "JaringanAng":  true,
          "MedcrevAng":   true,
          "DataAng":      true,
     }

     // "BPH":          true,
     var ValidSosmed = map[string]bool{
          "instagram": true,
          "linkedin":  true,
          "github":    true,
     }
```

- GET /api/v1/admin/pengurus
    - get all pengurus by division
    - role that can access these resource
        - superadmin
        - koor
        - bph
- DELETE /api/v1/admin/pengurus/:id
    - delete data pengurus
    - rola that can access these resource
        - superadmin
        - koor
        - bph

- Gallery
  managemen file foto
    - GET /api/v1/gallery?start_year=2024&end_year=2025

        - get all gallery and get gallery by filter year
            - if u want get all gallery just hit the api with no param
            - if u want to use the year filter send query param like this
                - start_year (format: YYYY) -> e.g: 2024
                - end_year (format: YYYY) -> e.g: 2025
        - there is pagination to
            - page (optional, integer, default=1)
            - limit (optional, integer, default=10)
        - public route
    - POST /api/v1/admin/gallery
        - insert gallery
        - role that can access these resource
            - Koor medcrev
            - superadmin
        - request body
            - gallery_name -> required -> string
            - gallery_type -> required -> string
            - description -> required -> string
            - event_date -> required -> (format: RFC3339) -> e.g: 2025-04-01T10:00:00+07:00
        - u can upload multiple file or single file image
            - max multiple upload -> 10 file (mboh servere tahan po ora ki hehe :_))
            - max file size -> 5mb
    - DELETE /api/v1/admin/gallery/:id
        - delete file image
        - request param
            - id
        - role that can access theese resource
            - koor medcrev
            - superadmin
- Blog
  manajemen blog
    - GET /api/v1/blogs/:id
        - get blog by id
        - request param -> id
        - public api
    - GET /api/v1/blog
        - get all blog and by kategori
        - pagination
            - page (optional, integer, default=1)
            - limit (optional, integer, default=10)
        - if no parameters are provided
            - get all blog
        - if any parameter are provided
            - get blog by kategory
                - parameter -> kategori
                - maksimal filter -> 3
                - daftar filter valid

                - event
                - seminar
                - collaboration
                - education
                - technology
                - work
                - activity
            - example
                - /api/v1/blog?kategori=event&kategori=seminar
    - public route
- GET /api/v1/admin/blog
    - get all blog and by kategori
    - pagination
        - page (optional, integer, default=1)
        - limit (optional, integer, default=10)
    - if no parameters are provided
        - get all blog
    - if any parameter are provided
        - get blog by kategory
            - parameter -> kategori
            - maksimal filter -> 3
            - daftar filter valid
                - event
                - seminar
                - collaboration
                - education
                - technology
                - work
                - activity
        - example
            - /api/v1/blog?kategori=event&kategori=seminar
        - role that can access these resource
            - superadmin
            - koor medcrev
- GET /api/v1/blogs/:id
    - get blog by id
    - request param -> id
    - public api
- PUT /api/v1/admin/blogs
    - update blog -> full update or partial update
    - request body
        - existingID_image -> array int
        - title -> string
        - slug -> string
        - content -> string
        - kategori -> array string
            - valid kategori
                - event

                - seminar
                - collaboration
                - education
                - technology
                - work
                - activity
            - status -> string
            - published_at -> (format: RFC3339) -> e.g: 2025-04-01T10:00:00+07:00
        - role that can access these resource
            - super admin
            - koor medcrev
    - DELETE /api/v1/admin/blogs/:id
        - delete blog by id
        - request param
            - id
        - role that can access these resource
            - super admin
            - koor medcrev
- Super_Admin
    - /api/v1/admin/ -> create new user
        - required param
            - email
            - fullname
            - role
    - /api/v1/admin/:id -> get user by id
    - /api/v1/admin/ -> get all user
        - get all users data except super_admin
    - /api/v1/admin/:id -> update user by id
    - /api/v1/admin/superadmin -> create superadmin (just use once)
- Koor & bph
    - /api/v1/koor/ -> create user
        - required param
            - email
            - fullname
        - can only create for its member
    - /api/v1/koor/:id -> delete user by id
        - same like get all user
    - /api/v1/koor/:id -> update user by id
        - same like get all user
    - /api/v1/koor/ -> get all user
        - can only get data from their division
    - /api/v1/koor/:id -> get user by id
        - same like get all user -> can only get data from their division
- pengurus
    - /api/v1/pengurus/ -> create pengurus -> need auth
        - user can access
            - all users
        - how it works

                - if u create ur own data, u don't need to fill in this field
                    - id_user
                    - email
                - fill in this field if u are creating data for somenone else
                    - id_user
                    - email
                - sosmed field is optional so u can fill in or not
        - /api/v1/pengurus/:id -> get pengurus by id -> public api
        - /api/v1/pengurus/:id -> update pengurus by id -> need auth
        - /api/v1/pengurus/ -> get all pengurus -> public api
            - has a filter
                - filtered by division
                - only returns data according to given filter
                - filter divisi
                    - "pemro" -> pemrograman
                    - "bph" -> pengurus harian
                    - "jaringan" -> jaringan
                    - "medcrev" -> crevmed
                    - "data" -> data
        - /api/v1/pengurus/:id -> Delete user by id -> need auth -> only koor and super admin can acces
- gallery -> only accessible by crevmed (dhion)
    - /api/v1/gallery/ -> insert gallery
        - you can insert one or more image file
    - -> get gallery by id
    - /api/v1/gallery/ -> get all gallery -> public api -> still have some bug
        - has a filter and pagination
            - filtered by type
                - only returns data according to given type
            - only returns 15 picture at a time
    - -> update gallery by id
    - -> delete gallery by id
- blog -> hanya bisa di akses oleh crevmed koor
- work

AUTH

- using jwt -> jwt save in cookies
- format cookie nya
    - Name: cfg.Name
    - Value: cfg.Value -> jwt
    - Path: cfg.Path
    - Expires: cfg.Expires -> 1 jam
    - Secure: cfg.Secure
    - HttpOnly: cfg.HttpOnly
    - SameSite: cfg.SameSite
- Cookie expired -> one hour

Struktur

```
 .
├──  bin
├──  cmd
│   ├──  api
│   │   └──  main.go   -> execute program/main program
│   └──  migrate
│       └──  main.go
├──  docs   -> all about documentation API
│   ├──  swagger_handler
│   │   └──  user.go
│   ├──  docs.go
│   ├──  swagger.json
│   └──  swagger.yaml
├──  internal   -> all logic BackEnd
│   ├──  auth   -> auth logic
│   │   ├──  handler.go    -> login and logout
│   │   ├──  middleware.go -> set jwt, cookie and get cookie
│   │   └──  service.go    -> logic to make jwt, hash and another thing
│   ├──  config
│   │   └──  env.go  -> load env file
│   ├──  database    -> all about database
│   │   ├──  model   -> all model structur and another things
│   │   │   ├──  blog.go
│   │   │   ├──  gallery.go
│   │   │   ├──  pengurus.go
│   │   │   ├──  users.go
│   │   │   └──  work.go
│   │   ├──  database.go     -> connection to database
│   │   └──  model.go        -> define all model so u can use them
│   ├──  handler   -> to handler http req/res from client
│   │   ├──  blog.go
│   │   ├──  gallery.go
│   │   ├──  pengurus.go
│   │   ├──  user.go
│   │   └──  work.go
│   ├──  routes -> all routes are specified here
│   │   ├──  route  -> the route of each model is defined here
│   │   │   ├──  auth_route.go
│   │   │   ├──  blog_route.go
│   │   │   ├──  gallery.go
│   │   │   ├──  pengurus_route.go
│   │   │   ├──  user_route.go
│   │   │   └──  work_route.go
│   │   └──  routes.go  -> a collection of all routes called here
│   ├──  server -> about server setup
│   │   └──  server.go -> setup ur server manually here
│   └──  service -> logic for handler
│       ├──  gallery_service.go
│       └──  pengurus_service.go
├──  migrations  -> file migration to set up database
│   ├──  000001_users_table.down.sql
│   ├──  000001_users_table.up.sql
│   ├──  000002_pengurus_table.down.sql
│   ├──  000002_pengurus_table.up.sql
│   ├──  000003_gallery_table.down.sql
│   ├──  000003_gallery_table.up.sql
```

```
│   ├── 000005_work_table.down.sql
│   ├── 000005_work_table.up.sql
│   ├── 000006_blog_table.down.sql
│   └── 000006_blog_table.up.sql
├── storage  -> place to save ur scandal video or photo (pm scandal :0)
│   └── uploads
│       ├── image
│       │   └── 17.png_achievment
│       └── video
├── tmp
│   ├── build-errors.log
│   └── main
├── web -> ini salah bikin, iyo iyo ngko tak resike
├── .air.toml  -> config air (hot reload)
├── .env -> file env ku dewe a
├── .env.example -> koe nggo ne seng iki
├── go.mod
├── go.sum
├── package-lock.json
├── package.json
├── Readme.md
└── README.md
```

License

MIT↵

---

run docker for minio

```
docker run -d -p 9000:9000 -p 9001:9001 --name minio -e MINIO_ROOT_USER=admin
-e MINIO_ROOT_PASSWORD=password123 minio/minio server /data --console-address
":9001"
```

```
const (
    RoleKeyPemroAnggota    = "pemroAnggota"
    RoleKeyJaringanAnggota = "jaringanAnggota"
    RoleKeyMedcrevAnggota  = "medcrevAnggota"
    RoleKeyDataAnggota     = "dataAnggota"
    RoleKeyBphAnggota      = "BPHAnggota"
    RoleKeyKoorPemro       = "KoorPemro"
    RoleKeyKoorJaringan    = "KoorJaringan"
    RoleKeyKoorData        = "KoorData"
    RoleKeyKoorMedcrev     = "KoorMedcrev"
    RoleKeyBPH             = "BPH"
    RoleKeySuperAdmin      = "SuperAdmin"
)
```